

1. Estrategia de Persistencia

Innovatech Solutions implementa el patron Database per Service, donde cada microservicio tiene su propia base de datos PostgreSQL completamente aislada. Esto garantiza que los microservicios sean independientes: si la base de datos de tareas necesita mantenimiento o migracion, los servicios de proyectos y equipos no se ven afectados.

La persistencia se implementa mediante Prisma ORM v5.22.0 (version fija sin ^), que proporciona tipado estricto end-to-end con TypeScript. Los modelos de datos se definen en archivos schema.prisma y las migraciones se gestionan con prisma migrate.

2. Bases de Datos

Microservicio	Base de Datos	Puerto (dev)	Usuario	Motor
ms-projects	db_projects	5434	user_projects	PostgreSQL 15
ms-tasks	db_tasks	5433	johndoe	PostgreSQL 15
ms-teams	db_teams	5435	user_teams	PostgreSQL 15

3. Modelos de Datos (Prisma Schema)

3.1 Project (ms-projects)

```
id: String @id @default(uuid())
name: String
description: String
status: String @default("PLANNED")
createdAt: DateTime @default(now())
updatedAt: DateTime @updatedAt
```

Estados validos: PLANNED, IN_PROGRESS, COMPLETED, ON_HOLD, CANCELLED

3.2 Task (ms-tasks)

```
id: String @id @default(uuid())
name: String
description: String?
area: String
status: TaskStatus @default(PENDING)
assignedTo: String
teamId: String
projectId: String
createdAt: DateTime @default(now())
updatedAt: DateTime @updatedAt
```

Estados (enum): PENDING, IN_PROGRESS, COMPLETED, BLOCKED

3.3 Team (ms-teams)

```
id: String @id @default(uuid())
name: String
description: String
status: TeamStatus @default(ACTIVE)
leaderId: String @default(uuid())
area: String
createdAt: DateTime @default(now())
updatedAt: DateTime @updatedAt
```

Estados (enum): ACTIVE, INACTIVE

4. Acceso a Datos (Patron Service Layer)

El acceso a la base de datos esta centralizado en la capa Service de cada microservicio. Ningun otro componente (Controller, Factory, Middleware) accede a Prisma directamente. El Service importa PrismaClient y ejecuta las operaciones CRUD:

Operacion	Metodo Prisma	Ejemplo
Crear	<code>prisma.model.create({ data })</code>	Crear proyecto con estado PLANNED
Leer todos	<code>prisma.model.findMany()</code>	Listar todos los proyectos
Leer por ID	<code>prisma.model.findUnique({ where: { id } })</code>	Obtener proyecto por UUID
Actualizar	<code>prisma.model.update({ where, data })</code>	Cambiar estado de tarea
Eliminar	<code>prisma.model.delete({ where: { id } })</code>	Eliminar equipo por ID

5. Migraciones

Las migraciones se gestionan con Prisma Migrate. Cada microservicio tiene su carpeta `prisma/migrations/` con el historial completo de cambios en el schema. Al ejecutar `docker compose up --build`, las migraciones se aplican automaticamente durante el build del contenedor (`npx prisma generate + npx prisma migrate deploy` en produccion).

Para desarrollo local: `npx prisma migrate dev --name descripcion_del_cambio`

6. Configuracion de Conexion

Cada microservicio configura su conexion a la base de datos mediante la variable de entorno `DATABASE_URL` definida en el `docker-compose.yml`:

```
DATABASE_URL=postgresql://usuario:password@nombre-contenedor:5432/nombre_bd
```

En Docker, los microservicios se conectan usando el nombre del contenedor de la base de datos (ej: `db-projects`) en lugar de `localhost`, ya que Docker DNS resuelve automaticamente los nombres de servicio a IPs internas.

7. Seguridad de Datos

Las bases de datos en el entorno de desarrollo exponen puertos al host (5433, 5434, 5435) para facilitar las migraciones y depuracion. En produccion con Docker Swarm, las bases de datos NO exponen puertos y solo son accesibles desde la red overlay interna del cluster. Los passwords de las bases de datos se configuran como variables de entorno en el compose, nunca hardcoded en el

codigo fuente.